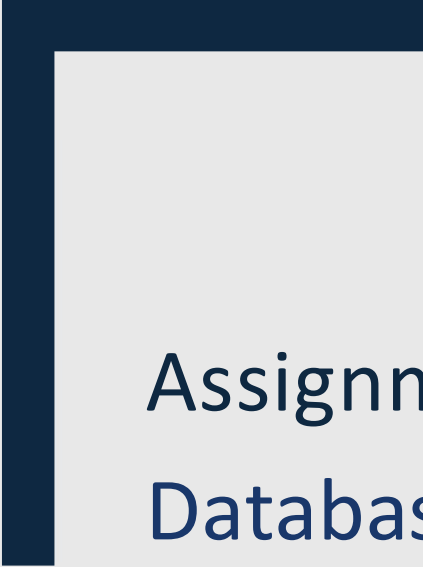




Assignment 2

Database Concepts 161

Themba Joy Mnisi 603766
Philisiwe Thabede 578194
Tebatso Moholola 603512

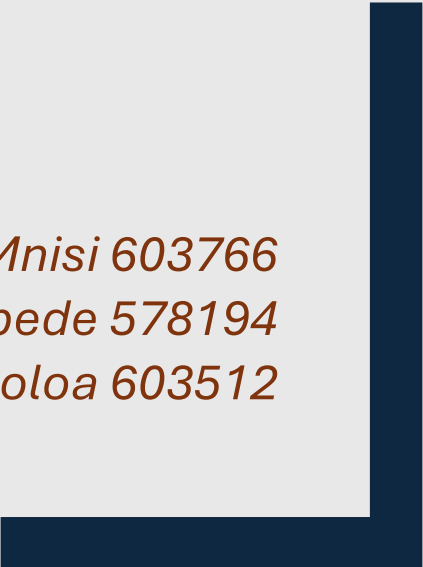


Table of Contents

Question 1 - ERD..... 2

Question 2..... 3

 a. Anomalies in Databases 3

 b. Identifying Functional Dependencies & Drawing a Dependency Diagram 4

 Problems in ONF: 7

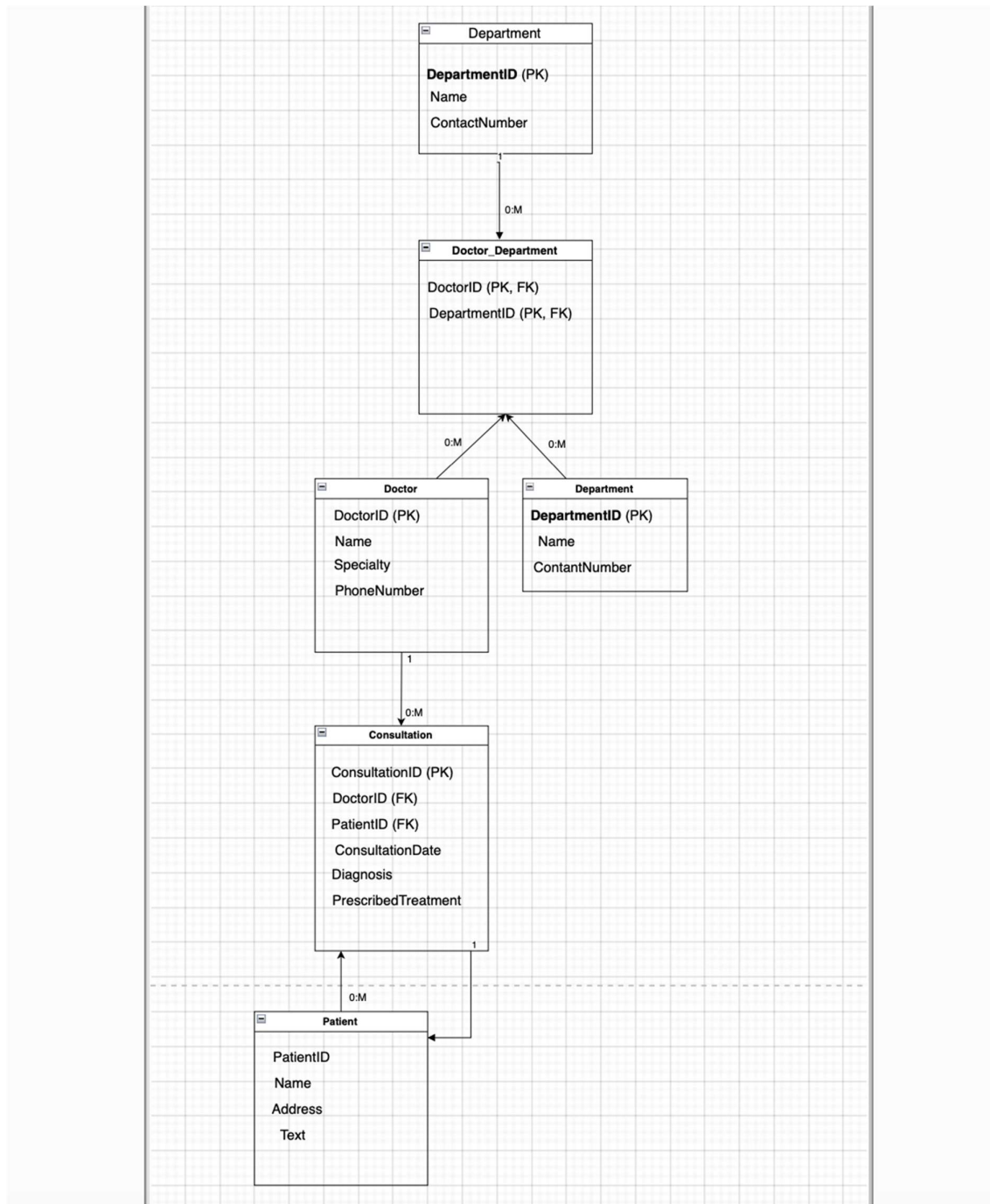
 First Normal Form (1NF) 7

 Second Normal Form (2NF) 8

 Step 4: Third Normal Form (3NF) 9

Bibliography..... 10

Question 1 - ERD



Department ↔ Doctor:

Departments are connected to doctors via the **Doctor_Department** associative entity, representing a many-to-many relationship.

Doctor ↔ Patient:

Doctors and patients are linked through the **Consultation** entity (which also holds additional consultation details), representing a many-to-many relationship.

Question 2

a. Anomalies in Databases

Databases help us store, organize, and manage data in a way that makes it easy to retrieve and update information. However, if the database is not well-structured, errors and problems can happen when we try to insert, delete, or update data. These problems are called database anomalies.

The table given in the question records customer car hire data for XYZ Rental. It includes details about cars, customers, hire dates, and rental offices. However, the way this table is structured makes it prone to errors.

There are three types of anomalies that can occur in a poorly designed database:

1. Insertion Anomaly – Problems when adding new data.
2. Deletion Anomaly – Problems when removing data.
3. Modification Anomaly – Problems when updating data.

Insertion Anomaly

An insertion anomaly happens when new information cannot be added unless extra, unnecessary information is also added.

Example:

XYZ Rental buys a new car, "M789 XYZ" (Ford Focus). The company wants to store this car in the system and assign it to the Brakenfell Office.

Problem:

- The table only stores a car when a customer hires it.
- Since no one has rented this car yet, it cannot be added to the table.
- The company would have to create a fake hire record, which is incorrect.

2. Deletion Anomaly

A deletion anomaly happens when removing one piece of information also removes other important details.

Example:

Look at this row from the table:

- Pen, T (C295) rented "M611 OPQ" (Nissan Tiida) from Durbanville Office on 20/5/98.
- Now, suppose Pen, T is no longer a customer, and the company deletes this rental record.

Problem:

- The car M611 OPQ is only recorded in this row.
- If this row is deleted, the car disappears from the system.
- But the car still exists and should be available for rental!

3. Modification Anomaly (Mistakes When Changing Data)

A modification anomaly happens when changing one piece of information requires multiple updates, which can cause mistakes.

Example 1: Changing Office Information

The Brakenfell Office (Office No: 01) appears many times in the table.

Problem:

- If the office changes its name or location, the company must update every row where "Brakenfell" appears.
- If even one row is missed, some records will have the old office name, while others will have the new office name.
- This makes the data confusing and incorrect.

Example 2: Changing Car Model

- The car "M565 OGD" (Ford Fiesta) appears twice in the table.
- Suppose the company changes the model name from "Fiesta" to "Fiesta XR".
- Every row where this car appears must be updated.
- If one row is missed, some records will say "Fiesta," and others will say "Fiesta XR," which is wrong and confusing.

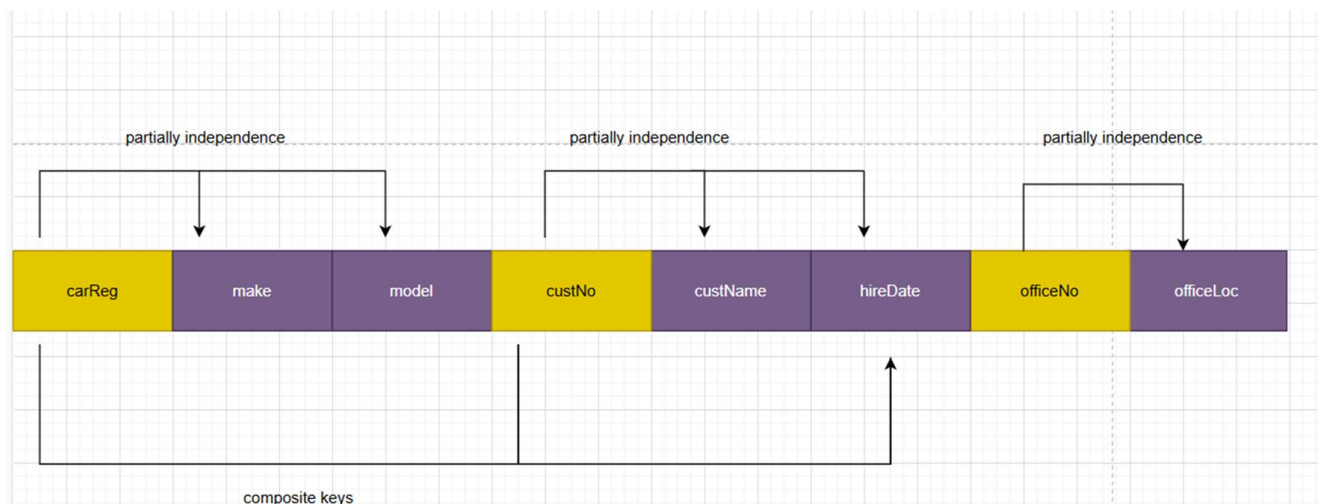
b. Identifying Functional Dependencies & Drawing a Dependency Diagram

A functional dependency is a rule that helps us understand how one piece of information can determine another. For example, if you know a car's registration number, you can figure out its make and model. Similarly, if you know a student's ID, you can find out their name. In the table we're looking at, we'll identify these kinds of rules to see how the data is connected. This will help us organize the information better and make sure it's accurate and consistent.

1. carReg = make, model

- Rule: If you know the car's registration number (carReg), you can figure out its make and model.
- Because each car has a unique registration number, and that number tells you what kind of car it is. For example, "M565 OGD" is always a Ford Fiesta.
- custNo = custName
 - If you know the customer's ID number (custNo), you can figure out their name.
 - Because each customer has a unique ID, and that ID tells you their name. For example, "C100" is always "Smith, J".
- officeNo = officeLoc
 - If you know the office number (officeNo), you can figure out its location.
 - Because each office has a unique number, and that number tells you where the office is. For example, "01" is always in Brakenfell.
- (carReg, custNo) = hireDate
 - If you know the car's registration number and the customer's ID number, you can figure out the hire date.
 - Because the combination of the car and the customer tells you when they hired the car. For example, "M565 OGD" and "C100" were hired on "14/5/98".

Diagram:



Justification for Functional Dependencies

This diagram shows how functional dependencies work in the given data. Some attributes depend only on part of the key, which causes partial dependency

1. carReg = make, model

A car's registration number (carReg) decides its make and model.

If carReg = "M565 OGD", then it must be Ford Fiesta.

A car registration number is unique for each car, so it always has the same make and model.

If we store the make and model in the same table as rentals, it repeats for every rental.

2. custNo = custName

A customer number (custNo) decides the customer name.

If custNo = "C100", then the customer is Smith, J.

A customer ID is unique, so it always belongs to the same customer.

If we store the customer name in the rental table, it repeats every time they rent a car.

3. officeNo = officeLoc

What it means: An office number (officeNo) decides the office location.

If officeNo = "01", then the location is Brackenfell.

Each office has a unique number, which never changes.

If we store the office location in the same table as rentals, it repeats for every rental from that office.

4. (carReg, custNo) → hireDate

A combination of carReg and custNo decides the hire date.

If carReg = "M565 OGD" and custNo = "C100", then the hire date is 14/5/98.

The hire date depends on which car was rented by which customer.

Storing hireDate in the same table as cars, customers, and offices makes it messy and inefficient.

c. Normalization Process

Unnormalized Form (ONF)

carReg	make	model	custNo	custName	hireDate	officeNo	officeLoc
M565 OGD	Ford	Fiesta	C100	Smith, J	14/5/98	01	Brakenfell
M565 OGD	Ford	Fiesta	C201	Hen, P	15/5/98	01	Brakenfell
N734 TPR	Nissan	Tiida	C100	Smith, J	16/5/98	01	Brakenfell
M134 BRP	Ford	Fiesta	C313	Blatt, O	14/5/98	02	Durbanville
M134 BRP	Ford	Fiesta	C100	Smith, J	20/5/98	02	Durbanville
M611 OPQ	Nissan	Tiida	C295	Pen, T	20/5/98	02	Durbanville

Problems in ONF:

- **Repeating groups:** A car is recorded multiple times for different hires.
- **Multivalued dependencies:** A customer can hire multiple cars, and each car can have multiple customers.
- **No clear primary key.**

First Normal Form (1NF)

- Each column contains atomic values (no repeating or grouped data).
- Each row is uniquely identifiable.
- **Primary Key Candidate: (carReg, custNo, hireDate)**

Second Normal Form (2NF)

To be in **2NF**, the table must be in **1NF** and all **partial dependencies** must be removed. The composite key is (**CarReg, CustNo, HireDate**).

Breaking the table into 2NF Relations:

Car Table (CarReg as Primary Key)

CarReg	Make	Model	OfficeNo
M565 OGD	Ford	Fiesta	01
N734 TPR	Nissan	Tiida	01
M134 BRP	Ford	Fiesta	02
M611 OPQ	Nissan	Tiida	02

Office Table (OfficeNo as Primary Key)

OfficeNo	OfficeLoc
01	Brankenfell
02	Durbanville

Customer Table (CustNo as Primary Key)

CustNo	CustName
C100	Smith, J
C201	Hen, P
C313	Blatt, O
C295	Pen, T

Hire Table (Composite Primary Key: CarReg, CustNo, HireDate)

CarReg	CustNo	HireDate	OfficeNo
M656 OGD	C100	14/5/98	01
M656 OGD	C201	15/5/98	01
N734 TPR	C100	16/5/98	01
M134 BRP	C313	14/5/98	02
M134 BPR	C100	20/5/98	02
M611 OPQ	C295	20/5/98	02

Now, all **partial dependencies** are removed.

Step 4: Third Normal Form (3NF)

To achieve **3NF**, we must remove **transitive dependencies**

- **OfficeLoc** depends on **OfficeNo**, not on the primary key of any table.
- **CustName** depends on **CustNo**, not on the primary key of any table.

Since we have already separated **Office** and **Customer** into their own tables, **3NF is achieved**.

1. **Car Table (CarReg as Primary Key)**
 - (CarReg, Make, Model, OfficeNo [FK])
2. **Office Table (OfficeNo as Primary Key)**
 - (OfficeNo, OfficeLoc)
3. **Customer Table (CustNo as Primary Key)**
 - (CustNo, CustName)
4. **Hire Table (CarReg, CustNo, HireDate as Composite Key)**
 - (CarReg [FK], CustNo [FK], HireDate, OfficeNo [FK])

Bibliography

<https://vertabelo.com/blog/er-diagram-for-hospital-management-system/>

<https://opentextbc.ca/dbdesign01/chapter/chapter-12-normalization/>